

Sal: Formal Reference for the Paper Revision

Concrete Equality, Certified Executions, and Independent Histories

FP Launchpad

7 October 2026

Abstract

This reference states the formal objects needed to revise the Sal paper: operation policies and event guards, commutation on valid concrete states, issuance-certified execution, represented merge conditions, and an independent sequential-history criterion. It is written for readers of the paper and does not require knowledge of Lean. A query must be explained by one history that contains exactly the original events, respects the required implementation and specification orders, and is admitted by the sequential specification. The merge theorem and the specification bridge are separate obligations.

Status and reading guide. The generic metatheory described here is machine-checked under its stated premises. Those premises remain obligations of each datatype; the theorem is not a claim that every MRDT satisfies them. The broken-remove example explains why specification visibility changes the paper’s literal criterion. The anchored queue and Fugue examples delimit the successful and unsuccessful applications of the stronger criterion.

The reference corresponds to the formalization on Sal branch `paper1`, commit `07fe525`, and was compared with the Overleaf manuscript snapshot `f3afb8c` of 7 October 2026. It does not certify physical networking or an implementation outside the stated mathematical machine. Numbered definitions and results are the targets of the separate manuscript-reconciliation note; Appendix A supplies optional mechanization traceability. Concrete equality is used throughout implementation reasoning. A specification simulation function, when used, has a separate purpose.

Contents

1	Semantic interfaces and the certified criterion	2
2	The execution model and recursive merge bases	5
3	Concrete representations and merge induction	6
4	Independent specification and history bridges	10
4.1	A sufficient global route	10
4.2	Choosing a permitted history within a certified execution	11
A	Mechanization traceability	13

1 Semantic interfaces and the certified criterion

The reference question is whether a merged implementation state has an explanation in an independently defined sequential language, using exactly its original events. Agreement of implementation replays alone does not answer this question. The definitions below describe the checked concrete and invariant-scoped interfaces. They distinguish the globally guarded sufficient class from the certified, represented-prefix class. An implementation need not inhabit the former to have a certificate in the latter.

Definition 1.1 (Events, effects, and payload policy). An implementation has a concrete state space Σ , initial state σ_0 , operations O , queries Q , answers V , a total effector $u : \Sigma \times (\mathbb{N} \times \mathbb{N} \times O) \rightarrow \Sigma$, a query read $: \Sigma \times Q \rightarrow V$, and a three-way merge $m : \Sigma^3 \rightarrow \Sigma$. An event is $e = (t, r, o)$, with projections $e.t, e.r, e.o$. For a list π , $u^*(s, \pi)$ is its left fold from s . A payload policy is a relation $P \subseteq O \times O$; write $a P b$ for $P(a.o, b.o)$. It does not inspect the events' external timestamps or replica identifiers. A replay context supplies eligible events $\mathcal{E}_C$ and visibility $\prec_C$. Set

$$a \parallel_C b \iff \neg(a \prec_C b) \wedge \neg(b \prec_C a), \quad \text{distinct}(a, b) \iff a.t \neq b.t.$$

Concrete raw commutation is

$$a \bowtie b \iff \forall s \in \Sigma, \quad u(u(s, a), b) = u(u(s, b), a).$$

These are relations on full events, even though P acts on payloads.

Definition 1.2 (Globally guarded replay laws). The global guarded sufficient class requires the following, for full events:

$$\begin{aligned} a.t \neq b.t \wedge a.r \neq b.r &\implies (\neg(a \bowtie b) \iff aPb \vee bPa), \\ a.t \neq b.t \wedge b.t \neq c.t &\implies \neg(aPb \wedge bPc). \end{aligned}$$

For every $s \in \Sigma$, intervening event list β , and a, b, c with pairwise distinct timestamps, it also requires

$$aPb \wedge \neg(b \bowtie c) \implies u(u^*(u(u(s, b), a), \beta), c) = u(u^*(u(u(s, a), b), \beta), c).$$

There is no different-replica guard on the last equation. Its absorber premise is actual noncommutation, rather than the policy edge bPc .

The no-chain clause allows $a = c$ when the two adjacent timestamp guards hold; therefore a payload self-edge is not rescued by external event freshness. These global equations quantify over arbitrary concrete scratch states and intervening lists. They are not the scoped laws below.

Definition 1.3 (Invariant-scoped concrete commutation and closure). For a fixed concrete state predicate $I : \Sigma \rightarrow \text{Prop}$, define

$$a \bowtie_I b \iff \forall s \in \Sigma, \quad I(s) \implies u(u(s, a), b) = u(u(s, b), a).$$

An invariant closure certificate at C supplies

$$\begin{aligned} &I(\sigma_0), \\ &\forall s, e, \quad I(s) \wedge e \in \mathcal{E}_C \implies I(u(s, e)), \\ &\forall l, a, b, \quad I(l) \wedge I(a) \wedge I(b) \implies I(m(l, a, b)). \end{aligned}$$

Commutation here has no simultaneous-readiness or reissuability premise. Closure includes replaying an already issued event out of causal order and repeating it. A port must independently justify the predicate and its closure; it cannot exclude a reachable state merely because that state refutes an equation.

Definition 1.4 (Version-local invariant order). For $E \subseteq \mathcal{E}_C$, use the *same* predicate I in both noncommutation tests:

$$\begin{aligned} a \text{ lo}_{C,E}^I b \quad &\Longleftrightarrow \quad (a \prec_C b \wedge \neg(a \bowtie_I b)) \\ &\vee (a \parallel_C b \wedge aPb \wedge \neg\exists c \in E. b \prec_C c \wedge \neg(b \bowtie_I c)). \end{aligned}$$

Only events in E can suppress a concurrent policy edge as absorbers. For $I(s) \equiv \text{True}$ this is the raw order $\text{lo}_{C,E}$, using $\bowtie$ throughout. A list enumerates E when it has no duplicates and its membership is exactly E . It respects a relation when every related pair in the list appears in that relation's direction.

Neither order is the full visibility relation. In particular, causal implementation-commuting events may be reordered by this order. Readiness adds causal constraints to implementation replay; it does not silently redefine the public semantic order.

Definition 1.5 (Original issuance, certified executions, and readiness). An issuance discipline is a predicate $G(e, s)$ checked at the materialized replica head from which e was minted. Mint honesty at C requires, for every $e \in \mathcal{E}_C$, a list π_e satisfying

$$\pi_e \text{ enumerates } \{p \in \mathcal{E}_C \mid p \prec_C e\}, \quad \pi_e \text{ respects } \prec_C, \quad G(e, u^*(\sigma_0, \pi_e)).$$

An issued application is an actual raw store application with its original head/version evidence and $G(e, s)$; non-application steps preserve the raw fork/merge rule. A certified execution starts at the initial store and, at every transition, retains mint honesty before and after the issued step. The widened execution permits recursively synthesized virtual merge bases. It has its own certified reachability relation, with the ordinary steps as its base fragment.

A replay scope is (C, F, R) , where F is its fixed eligible event set and $R(H, s)$ records representation of the already replayed prefix H . Define

$$\text{Ready}(H, e) \quad \Longleftrightarrow \quad e \in F \wedge e \notin H \wedge \forall p \in F, p \prec_C e \implies p \in H.$$

A replay is legal from H by the inductive rules

$$\text{Legal}(H, []), \quad \frac{\text{Ready}(H, e) \quad \text{Legal}(H \cup \{e\}, \pi)}{\text{Legal}(H, e :: \pi)}.$$

The issuance guard is retained as evidence about the original mint. It is not checked again when the effector is replayed at a different scratch state.

Definition 1.6 (Scoped replay and policy obligations). The invariant replay laws require

$$R(H, s) \implies I(s), \quad R(H, s) \wedge \text{Ready}(H, e) \implies R(H \cup \{e\}, u(s, e)).$$

They also require the concrete diamond $u(u(s, a), b) = u(u(s, b), a)$ whenever $R(H, s)$, both events are ready, $a \parallel_C b$, and neither direction of $\text{lo}_{C,F}^I$ orders them. The accompanying policy laws require

$$\neg(a \bowtie_I b) \Longleftrightarrow aPb \vee bPa$$

only for $a, b \in F$ that are concurrent, have distinct timestamps, and come from different replicas. No-chain is required for $a, b, c \in F$ with $a.t \neq b.t$ and $b.t \neq c.t$, as in Definition 1.2. The package consists of both replay laws and policy laws.

An alternative represented-prefix commutation test instead quantifies over represented prefixes where both events are ready. That definition is distinct from $\bowtie_I$; replacing the latter by readiness would make causal-pair tests potentially vacuous.

Theorem 1.7 (Scoped implementation replay equality). *Suppose the replay laws of Definition 1.6 hold, $R(H, s)$, and π_1, π_2 enumerate the same event set A . If both lists are legal from H and each respects both $\prec_C$ and $\text{lo}_{C,F}^I$, then*

$$u^*(s, \pi_1) = u^*(s, \pi_2).$$

Its hypotheses are scoped replay laws, not the global class of Definition 1.2. This theorem proves concrete implementation agreement. Independent sequential legality is a further obligation.

Definition 1.8 (Independent history language and contextual conflict). For an explicitly chosen update alphabet U , let labels be $\text{upd}(x)$, $x \in U$, and $\text{qry}(q, v)$. A history specification S is a prefix-closed set of finite label lists containing the empty list. A labelled transition system defines such a language by accepting every finite run from its initial state; no determinism or totality is assumed. Thus acceptance is a proposition, not a uniquely determined resulting state. Define contextual specification commutation by

$$\begin{aligned} x \bowtie_S y &\iff \forall \rho, \tau, \quad \rho \cdot [\text{upd}(x), \text{upd}(y)] \cdot \tau \in S \\ &\iff \rho \cdot [\text{upd}(y), \text{upd}(x)] \cdot \tau \in S. \end{aligned}$$

For a fixed label projection $f : (\mathbb{N} \times \mathbb{N} \times O) \rightarrow U$, let $\text{labels}_f(\pi)$ retain the original event order while mapping each update through f . The independent specification visibility relation is

$$a \text{sv}_{C,S}^{f,S} b \iff a \prec_C b \wedge \neg(f(a) \bowtie_S f(b)).$$

Full-input specifications use $f = \text{id}$. Payload projection $f(e) = e.o$ is an explicit interface choice. It cannot be assumed adequate for specifications that distinguish original enqueue or insertion identities.

Definition 1.9 (The full-event invariant criterion). For every allocated version v with stored record (s, E) and every query q , there must exist *one* list π such that

1. π enumerates exactly E ;
2. π respects $\text{lo}_{C,E}^I$;
3. π respects $\text{sv}_{C,S}^{\text{id},S}$;
4. $\text{labels}_{\text{id}}(\pi) \cdot [\text{qry}(q, \text{read}(s, q))] \in S$.

The universal invariant $I(s) \equiv \text{True}$ recovers the raw full-event criterion, including specification visibility. Checking every allocated version implies the corresponding guarantee for current replica heads. A projected criterion replaces the specification alphabet through f ; it still preserves the specification conflicts of those projected labels. The manuscript's literal criterion omits item 3 and uses payload labels. These are distinct criteria, rather than alternate notations for the same proposition.

An independent sequential simulation may use a relation, or an abstraction function $\alpha : \Sigma \rightarrow \Phi$, to relate implementation states to the *specification's* states. This does not replace concrete equality in the merge VCs, diamonds, or Definition 1.3. Universal fold soundness is one sufficient bridge. Certified Queue and RGA results instead establish appropriate original-event witnesses for represented histories. Neither implementation replay equality nor existence of an implementation fold alone establishes item 4, and a specification-legal witness must also satisfy item 3.

2 The execution model and recursive merge bases

This section specifies the store underlying certified execution in Definition 1.5. A merge base is a property of the version graph; its state is then used by the datatype's three-way merge.

Definition 2.1 (Ranked store and ordinary execution). A configuration is $C = (S, h, \text{par}, \prec_C)$, where $S(v) = (s_v, E_v)$ is a partial store, $h(r)$ is an active replica's allocated head, and $\text{par}(v)$ lists its parents. Version ranks are natural numbers and each parent has smaller rank. Version 0 stores $(\sigma_0, \emptyset)$. Visibility endpoints belong to the events represented at active heads. Visible predecessors of a head's events belong to that head, distinct events have distinct timestamps, and

$$a \prec_C b \implies a.t < b.t.$$

Distinct events from the same replica are visibility-comparable. Reachable stores additionally have transitive visibility and causally closed stored histories. Write $v \preceq w$ for the reflexive transitive closure of parent edges directed from parent to child.

All steps preserve the store except at the explicitly fresh version, preserve other heads, and leave visibility unchanged unless stated otherwise:

1. **Fork.** For a fresh replica r' and existing head $h(r) = v$, allocate $v' > v$ with $S(v') = S(v)$, $\text{par}(v') = [v]$, and $h(r') = v'$. The source head is unchanged.
2. **Apply.** For $h(r) = v$, $S(v) = (s, E)$, and $e = (t, r, o)$, allocate $v' > v$ with

$$S(v') = (u(s, e), E \cup \{e\}), \quad \text{par}(v') = [v], \quad h(r) = v'.$$

Replace visibility by $\prec_C \cup (E \times \{e\})$. The timestamp is fresh against all events in the registry and every stored history. The target configuration must satisfy the clock condition above, so $p.t < t$ for every $p \in E$. Raw apply has no client guard; certified apply adds $G(e, s)$ from Definition 1.5.

3. **Ordinary merge.** For heads v_1, v_2 with stored states s_1, s_2 , choose an allocated greatest common ancestor g as defined below. Allocate $v' > v_1, v_2$ with

$$S(v') = (m(s_g, s_1, s_2), E_{v_1} \cup E_{v_2}), \quad \text{par}(v') = [v_1, v_2].$$

Update the receiving replica's head to v' .

4. **Query.** Return $\text{read}(s_{h(r)}, q)$ without changing the configuration.

The initial store has just version 0 and one replica pointing to it. Executions are finite sequences of these steps. The certified relation also retains the original-mint evidence described in Definition 1.5.

Definition 2.2 (Greatest common ancestor). A greatest common ancestor g of v_1, v_2 satisfies

$$g \preceq v_1 \wedge g \preceq v_2 \wedge \forall w, (w \preceq v_1 \wedge w \preceq v_2) \implies w \preceq g.$$

A maximal common ancestor need only have no strictly later common ancestor; there may be several incomparable maximal common ancestors and no greatest one. The store model carries the coherence property $E_g = E_{v_1} \cap E_{v_2}$ whenever an allocated greatest common ancestor exists.

Theorem 2.3 (Intersection from event origins). *Suppose parents are allocated, histories grow along parent edges, and every event has an allocated origin version containing it which is an ancestor of every version containing that event. Then for allocated g, v_1, v_2 with g their greatest common ancestor,*

$$E_g = E_{v_1} \cap E_{v_2}.$$

The forward inclusion follows from history monotonicity. For a shared event, its origin is common to both heads and therefore precedes g , proving the reverse inclusion. The store-invariant proof supplies this justification for the coherence field of Definition 2.2; it is not a statement about arbitrary event labels attached to an arbitrary DAG.

Definition 2.4 (Recursive virtual merge base). For a finite set of versions T and a version w , let $M(T, w)$ be the list, in increasing rank, of maximal elements of

$$\{x \mid x \preceq w \wedge \exists v \in T, x \preceq v\}.$$

Write s_v for the stored state at v , using σ_0 as the total-function default when unallocated; correctness uses allocated supports. Define the base B and scratch fold F by

$$\begin{aligned} B(T, w) &= \begin{cases} \sigma_0, & M(T, w) = [], \\ F(\{v\}, s_v, \eta), & M(T, w) = v :: \eta, \end{cases} \\ F(T, s, []) &= s, \\ F(T, s, v :: \eta) &= F(T \cup \{v\}, m(B(T, v), s, s_v), \eta). \end{aligned}$$

The virtual base of v_1, v_2 is $B(\{v_1\}, v_2)$. The widened merge step uses this state in place of s_g in Definition 2.1, without requiring a greatest common ancestor. Only the final merged version is allocated. The scratch folds do not create replicas, events, or intermediate stored versions.

The recursion is well-founded by the size of the joint ancestor support, lexicographically with the pending list length. The execution theorem must establish correctness of these recursive scratch states as well as ordinary stored versions; replacing g by an arbitrary maximal common ancestor is not this construction.

3 Concrete representations and merge induction

The merge argument must retain enough information to reconstruct an event's metadata, including commuting predecessors when necessary. The following contract states the concrete equations used by the checked merge induction. It establishes representation preservation. An independent sequential-history argument is still required for the correctness criterion.

Write $u_e(s)$ for the update of state s by the original event e , and $m(l, a, b)$ for three-way merge. In this section only, $a \leftrightarrow b$ means commutation on *all* concrete states. For an operation policy P on update payloads, define the raw semantic order

$$\begin{aligned} \ell_H^C(a, b) \iff & (a \prec_C b \wedge \neg(a \leftrightarrow b)) \\ & \vee (\neg(a \prec_C b) \wedge \neg(b \prec_C a) \wedge P(a.o, b.o) \\ & \wedge \neg \exists c \in H. b \prec_C c \wedge \neg(b \leftrightarrow c)). \end{aligned}$$

These are the relations consumed by the raw merge theorem; they do not replace invariant-scoped commutation in the final history criterion.

Definition 3.1 (Indexed concrete representation). A representation is a predicate

$$R : \text{ReplayContext} \times \mathcal{P}(\mathcal{E}) \times \Sigma \longrightarrow \text{Prop}.$$

The assertion $R(C, H, s)$ relates the *entire concrete state* s to a specific event set H in context C . It need not be defined as reachability, and it is not inferred from the desired query answer. Define

$$\begin{aligned}\text{Supp}_C(H) &\iff H \subseteq \mathcal{E}_C, \\ \text{Unique}(R) &\iff \forall C, H, s, t. R(C, H, s) \wedge R(C, H, t) \implies s = t.\end{aligned}$$

An enumeration $\pi \equiv H$ is a finite list without repetitions whose members are exactly H . Thus every finite-set assumption below is an explicit enumeration assumption.

Definition 3.2 (Metadata dependencies). For every context C , choose a relation M_C on events satisfying

$$\begin{aligned}M_C(a, b) &\implies a \prec_C b, \\ a \prec_C b \wedge \neg(a \leftrightarrow b) &\implies M_C(a, b).\end{aligned}$$

It may retain additional visible predecessors which commute. Define

$$\begin{aligned}\text{Closed}_{M_C}(H) &\iff \forall a, b. M_C(a, b) \wedge b \in H \implies a \in H, \\ \text{Past}_{M_C}(e) &= \{x \mid x = e \vee M_C^+(x, e)\}, \\ \text{Max}_{C,H}^\ell(e) &\iff \forall x \in H \setminus \{e\}. \neg \ell_H^C(e, x), \\ \text{Max}_{C,H}^M(e) &\iff \forall x \in H \setminus \{e\}. \neg M_C(e, x).\end{aligned}$$

The two maximality conditions are separate. If $H \subseteq U$ is M_C -closed and e is M_C -maximal in U , then $H \setminus \{e\}$ remains M_C -closed. Semantic maximality alone does not justify this step. For the embedded sequence and anchored queue ports, $M_C = \prec_C$; their reconstruction past retains all causal metadata.

Definition 3.3 (Five concrete merge VCs). The following conditions are universally quantified over the displayed contexts, histories, events and states. Define the common hypotheses

$$\begin{aligned}A_C(H) &:= \text{Supp}_C(H) \wedge \text{Closed}_{M_C}(H), \\ K_C(H_1, H_2, e) &:= \text{Trans}(\prec_C) \wedge \text{Irrefl}(\prec_C) \wedge A_C(H_1) \wedge A_C(H_2) \\ &\quad \wedge \text{Max}_{C,H_1 \cup H_2}^\ell(e) \wedge \text{Max}_{C,H_1 \cup H_2}^M(e).\end{aligned}$$

Within each condition, abbreviate $R_C(H, s) := R(C, H, s)$, $D := \text{Past}_{M_C}(e)$ and $H^- := H \setminus \{e\}$. These abbreviations compress only repeated hypotheses.

VC1. Merge commutativity. From

$$A_C(H_1), A_C(H_2), R_C(H_1 \cap H_2, l), R_C(H_1, a), R_C(H_2, b)$$

conclude $m(l, a, b) = m(l, b, a)$. This equation is required on represented triples, not on arbitrary concrete states.

VC2. Initial branch. From $A_C(H)$ and $R_C(H, s)$ conclude

$$m(\sigma_0, \sigma_0, s) = s.$$

VC3. Local redistribution. Assume $K_C(H_1, H_2, e)$, $e \in H_1 \setminus H_2$, and *all* the representation premises

$$\begin{aligned}R_C(H_1 \cap H_2, l), \quad R_C(D^-, B), \quad R_C(H_1^-, t), \quad R_C(H_2, b), \\ R_C(D, u_e(B)), \quad R_C(H_1, m(B, t, u_e(B))), \\ R_C((H_1 \cup H_2)^-, m(l, t, b)).\end{aligned}$$

Then

$$m(l, m(B, t, u_e(B)), b) = m(B, m(l, t, b), u_e(B)).$$

VC4. Shared redistribution. Assume $K_C(H_1, H_2, e)$, $e \in H_1 \cap H_2$, and

$$\begin{aligned} & R_C((H_1 \cap H_2)^-, t_0), \quad R_C(D^-, B), \\ & R_C(H_1^-, t_1), \quad R_C(H_2^-, t_2), \quad R_C(D, u_e(B)), \\ & R_C(H_1 \cap H_2, m(B, t_0, u_e(B))), \\ & R_C(H_1, m(B, t_1, u_e(B))), \quad R_C(H_2, m(B, t_2, u_e(B))), \\ & R_C((H_1 \cup H_2)^-, m(t_0, t_1, t_2)). \end{aligned}$$

Then

$$\begin{aligned} & m(m(B, t_0, u_e(B)), m(B, t_1, u_e(B)), m(B, t_2, u_e(B))) \\ & = m(B, m(t_0, t_1, t_2), u_e(B)). \end{aligned}$$

VC5. Causal delta. Assume transitive, irreflexive visibility, $A_C(U)$, $e \in U$, both $\text{Max}_{C,U}^\ell(e)$ and $\text{Max}_{C,U}^M(e)$, and

$$R_C(U^-, s), \quad R_C(D^-, B), \quad R_C(D, u_e(B)), \quad R_C(U, u_e(s)).$$

Then $m(B, s, u_e(B)) = u_e(s)$.

The represented smaller-union merge in VC3 and VC4 is obtained by a *strictly smaller* induction call. Its presence is not a premise asserting the desired join theorem for the original union. The represented side reconstructions are also derived on proper subsets, or by VC5 when a side equals the whole union.

Definition 3.4 (Replay supply and a peel witness). At a fixed context C , replay supply consists of both clauses:

- (i) For every H and π , if $\pi \equiv H$ and $\text{Supp}_C(H)$, some s satisfies $R_C(H, s)$.
- (ii) For every represented, supported, nonempty, M_C -closed U , there exist $e \in U$ and states s, B satisfying

$$\begin{aligned} & \text{Max}_{C,U}^\ell(e), \quad \text{Max}_{C,U}^M(e), \\ & R_C(U^-, s), \quad R_C(\text{Past}_{M_C}(e)^-, B), \\ & R_C(\text{Past}_{M_C}(e), u_e(B)), \quad R_C(U, u_e(s)). \end{aligned}$$

The second clause is the *peel witness*. It supplies update reconstruction, not a merge-preservation assumption. Its existence is a separate datatype obligation; acyclicity of ℓ alone supplies neither metadata maximality nor the four representation assertions.

Theorem 3.5 (Finite concrete join induction). Assume the five VCs, $\text{Unique}(R)$, and

$$\begin{aligned} & \forall C, H, s. R(C, H, s) \implies R(C, \emptyset, \sigma_0), \\ & \forall C, H, s. R(C, H, s) \implies \exists \pi. \pi \equiv H. \end{aligned}$$

Fix a context C with transitive, irreflexive visibility and replay supply. For all H_1, H_2, l, a, b , if $A_C(H_1)$, $A_C(H_2)$ and

$$R_C(H_1 \cap H_2, l), \quad R_C(H_1, a), \quad R_C(H_2, b),$$

then

$$R_C(H_1 \cup H_2, m(l, a, b)).$$

Proof structure. Induct strongly on the length of a duplicate-free enumeration of $H_1 \cup H_2$. An empty side uses uniqueness, the initial-representation hypothesis and VC2, with VC1 if the right side is empty. Otherwise replay supply chooses a peel witness. Removing its event preserves metadata closure by M_C -maximality. Supply gives represented predecessor states for the branches and their intersection. The smaller induction calls establish side reconstruction and the smaller-union merge representation. Uniqueness identifies the reconstructed sides with the original sides; VC3 or VC4 then brings the event outside. VC5 and the peel witness's represented update reconstruct the union. Every recursive call has a strictly smaller event set; none assumes representation preservation of an unresolved merge. $\square$

Remark 3.6 (The full-causal-closure join contract). The checked derived contract quantifies over fully causally closed H_1, H_2 , not over all merely conflict-closed histories. To obtain it from Theorem 3.5, require replay supply for every C, H_1, H_2, a, b with transitive, irreflexive visibility, supported sides and $R_C(H_1, a), R_C(H_2, b)$. This is the quantifier order of the supply hypothesis in the checked derived theorem. Causal closure implies M_C -closure because $M_C \subseteq \prec_C$. Hence the theorem gives the concrete *representation join* contract

$$\begin{aligned} & \text{Supp}_C(H_1), \text{Supp}_C(H_2), \text{CausalClosed}_C(H_1), \text{CausalClosed}_C(H_2), \\ & R_C(H_1 \cap H_2, l), R_C(H_1, a), R_C(H_2, b) \implies R_C(H_1 \cup H_2, m(l, a, b)), \end{aligned}$$

under transitive, irreflexive visibility. This contract includes intermediate representation evidence which canonical query agreement by itself does not provide.

Theorem 3.7 (Lifting to certified executions). *Fix an internal replay policy whose directed event relation is ρ . This auxiliary policy supplies concrete replay witnesses; it is separate from the public payload policy and from the revised invariant-scoped history order. Let $K_\rho(a, b) := \rho(a, b) \vee \rho(b, a)$. Its set-relative order is*

$$\begin{aligned} \lambda_{C,H}^\rho(a, b) & \iff (a \prec_C b \wedge K_\rho(a, b)) \\ & \vee (\neg(a \prec_C b) \wedge \neg(b \prec_C a) \wedge \rho(a, b) \\ & \quad \wedge \neg \exists c \in H. b \prec_C c \wedge K_\rho(b, c)). \end{aligned}$$

Define

$$\text{Can}_C^\rho(H, s) \iff \exists \pi. \pi \equiv H \wedge \text{Respects}(\pi, \lambda_{C,H}^\rho) \wedge u^*(\sigma_0, \pi) = s.$$

Suppose, for every configuration C satisfying original mint honesty, the following full-causal-closure canonical join contract holds:

$$\begin{aligned} & \text{Trans}(\prec_C), \text{Irrefl}(\prec_C), \text{Supp}_C(H_1), \text{Supp}_C(H_2), \\ & \text{CausalClosed}_C(H_1), \text{CausalClosed}_C(H_2), \\ & \text{Can}_C^\rho(H_1 \cap H_2, l), \text{Can}_C^\rho(H_1, a), \text{Can}_C^\rho(H_2, b) \\ & \implies \text{Can}_C^\rho(H_1 \cup H_2, m(l, a, b)). \end{aligned}$$

Every ordinary or recursive-virtual issuance-certified execution from the initial configuration then has $\text{Can}_C^\rho(H, s)$ at every allocated version $C.S(v) = (s, H)$. Its event sets are supported and causally closed; visibility is transitive and irreflexive. Moreover, if the store invariant and this canonical configuration invariant hold, its recursive virtual base for versions (s_1, H_1) and (s_2, H_2) is canonical for $H_1 \cap H_2$.

The displayed join hypothesis is derived rather than supplied as a datatype axiom: the five VCs, uniqueness, initial representation, finite enumeration and replay supply yield representation join. A port additionally proves

$$R_C(H, s) \iff \text{Can}_C^\rho(H, s)$$

for supported, causally closed histories in each mint-honest context with transitive, irreflexive visibility. The backward implication makes the three canonical inputs represented; representation join constructs the union; the forward implication makes its result canonical. The embedded ports have exactly this correspondence. Other ports can use stronger represented canonical-state certificates rather than asserting this equivalence generally.

The execution induction carries the store invariant together with canonicity. Ordinary merges use the actual greatest common ancestor's intersection history. This event-intersection coherence is an admissibility property of the configuration in Definition 2.1; for operational stores it is proved from the store invariant, rather than from graph ancestry alone. Recursive virtual merges use a nested induction on finite ancestry support: the inner fold represents the union of its inputs, and each recursively computed base represents their intersection. Original issuance guards remain at original mint states. Neither intermediate peeling nor scratch computation requires reissuing an event. This theorem is a merge-and-replay guarantee; legality and specification visibility of one independent sequential word are additional obligations of the final correctness bridge.

4 Independent specification and history bridges

Representation correctness answers whether a stored state is explained by the issued event set. The independent specification answers whether that explanation is a permitted sequential history. These are separate obligations. In particular, convergence of implementation replays of the same history does not establish that any of their query answers belongs to the intended datatype.

Use the independent history language and projection of Definition 1.8. Write S for that language, retain its projection f , and abbreviate

$$F(\pi) = u^*(\sigma_0, \pi).$$

This expression contains no merges. Write $L_{C,E}$ for the implementation order chosen for context C and endpoint event set E : the raw order of Definition 1.4, or its separately justified invariant-scoped replacement. Its separate specification-visibility obligation uses $\text{sv}_C^{f,S}$ from Definition 1.8. Event identities remain in the witness even when the independent specification ignores their metadata.

4.1 A sufficient global route

A local simulation into an independent deterministic sequential machine M for payload labels consists of a relation R_{seq} , initial machine state m_0 , machine update w and query read_M , with these properties:

$$\begin{aligned} & R_{\text{seq}}(\sigma_0, m_0), \\ & R_{\text{seq}}(s, m) \implies R_{\text{seq}}(u(s, e), w(m, e.o)) && \text{for every event } e, \\ & R_{\text{seq}}(s, m) \implies \text{read}(s, q) = \text{read}_M(m, q) && \text{for every query } q. \end{aligned}$$

There is no issuance, freshness or invariant premise on the update clause. For $f(e) = e.o$, induction establishes the unrestricted sequential-history premise

$$\forall \pi, q, \quad \text{labels}_f(\pi) \cdot [\text{qry}(q, \text{read}(F(\pi), q))] \in S. \tag{1}$$

Together with a stored replay witness and the replay laws, this suffices for the literal criterion: choose the replay word and substitute its equality with the stored state. The unrestricted premise concerns *merge-free folds*; it does not assert that every permutation respects $L_{C,E}$ or is an admissible linearization.

For the stronger criterion, the global bridge additionally records commutation compatibility:

$$\text{Commutes}_{\text{impl}}(a, b) \implies f(a) \bowtie_S f(b). \quad (2)$$

Here implementation commutation must be the commutation notion used in the chosen order. By contraposition, every specification-conflicting visibility edge is then an implementation-conflicting visibility edge. The same replay word respects both relations. Equation (1) still supplies history admission; compatibility alone cannot supply it.

Example 4.1 (A convergent set with a no-op remove). Consider a one-element set implemented by a finite set of addition timestamps, initially empty. An add inserts its timestamp; a remove leaves the state unchanged; the query returns whether the timestamp set is nonempty. Retain the ordinary observed-remove three-way merge

$$m(l, a, b) = (l \cap a \cap b) \cup (a \setminus l) \cup (b \setminus l).$$

Let the independent sequential machine instead implement add as setting presence to true and remove as setting it to false.

The real, single-replica execution is

$$\text{add} ; \text{remove} ; \text{query}(\text{true}).$$

All implementation updates commute, so with an empty arbitration policy the literal implementation order is empty. The witness [remove, add] explains the wrong answer in the independent language and satisfies the literal criterion. Indeed, this mutant converges and satisfies the literal criterion on every raw ordinary execution; it does not satisfy unrestricted fold soundness.

In the independent set language, add and remove do not contextually commute. Their causal order is therefore required by $\text{sv}_C^{f,S}$. The only respecting witness is add before remove, whose query answer is false. The stronger criterion rejects the displayed execution. This is a certified counterexample to identifying literal RA with preservation of sequential specification conflicts; it is not a counterexample to a theorem that assumes Equation (1).

4.2 Choosing a permitted history within a certified execution

Global fold soundness can be unsuitable for a datatype whose public operations have guards: an arbitrary word may name a nonexistent birth, reuse an identity, or remove a live non-head. Restricting the implementation's representation invariant does not automatically prove the independent language's guards. A second route proves existence of a suitable history on the supported, causally closed event sets of certified executions.

Definition 4.2 (Execution-scoped history adequacy). Fix an issuance discipline G , an implementation order L , and an independent language S . They satisfy execution-scoped history adequacy when, for every G -certified configuration C , every stored version (s, E) and every query q , there is a word π such that

$$\begin{aligned} &\pi \text{ enumerates } E \text{ exactly once,} \\ &\pi \text{ respects } L_{C,E} \text{ and } \text{sv}_C^{f,S}, \\ &\text{labels}_f(\pi) \cdot [\text{qry}(q, \text{read}(F(\pi), q))] \in S. \end{aligned}$$

The quantifier order is $\forall q \exists \pi$. The accepted answer describes the merge-free fold, rather than the stored state s . Both order obligations and language admission use the same π .

Theorem 4.3 (Representation and history bridge). *Fix the datatype, issuance discipline, independent language, and chosen implementation replay order. Define exact concrete canonicity by*

$$\text{Can}_{C,E}(s) \iff \exists \pi. \pi \text{ enumerates } E \wedge \pi \text{ respects } L_{C,E} \wedge F(\pi) = s.$$

The concrete specialization of the history bridge has these assumptions:

1. *the chosen replay order has a proved uniqueness property: any two exact canonical states for the same supported event set agree on all queries;*
2. *every stored event set of the certified execution is supported, and its stored state is canonical for that set;*
3. *execution-scoped history adequacy of Definition 4.2 holds.*

Then every stored version satisfies the criterion with independent specification visibility: one word enumerates its original events, respects both required relations, and explains its stored query answer in S . Consequently all active heads satisfy that criterion. If the assumptions are established at every configuration of a finite certified execution, the result holds throughout that execution, including executions with recursively synthesized virtual merge bases.

Proof. Adequacy supplies a word π whose fold is canonical by its permutation and order properties. Replay uniqueness identifies the queries of that fold and the separately canonical stored state. Substitute this query equality in the accepted history. Its word, permutation and two order properties do not change. Apply the argument at each visited configuration for the execution claim. The stored-state premise is where the merge argument enters; history adequacy alone cannot discharge it. $\square$

This uniqueness assumption must be established for the chosen implementation order; it is not automatic from scoped laws. For example, Theorem 1.7 requires causally ready implementation replays; it does not give equality of every pair of permutations respecting only the invariant order. A concrete port may instead prove an exact representation relation between its selected independent history and the stored live state, as in Example 4.4.

Example 4.4 (Anchored enqueue with an independent FIFO). Enqueue records its issuer's observed tail and dequeue its observed head. The implementation keeps ordered live records and carried coordinates; it stores no removed birth records. The independent FIFO appends a fresh tagged value, ignoring its recorded tail; it removes a named head, permits repeated removal of an already removed identity, and rejects removal of a live non-head. The absent-removal case requires a prior birth in the sequential prefix; invented identities cannot be removed. Lack of tombstones does not mean lack of metadata: carried coordinates preserve placement information and can grow with their ancestry.

Head-only issuance implies that removed births form a causal downset among enqueue events. Order the original dequeue events chronologically. Immediately before an identity's first dequeue, emit its birth; emit subsequent duplicate dequeues without another birth. Append the surviving births in their stored coordinate order. The FIFO is empty between the removal blocks. The downset property preserves causal enqueue conflicts, and surviving coordinates preserve the remaining causal enqueue order. This one word has the exact final tagged live list and satisfies both order obligations. The resulting theorem applies to certified ordinary and recursive virtual executions with the unchanged public tagged-head query.

Example 4.5 (Fugue and a live-anchor specification). The original Fugue issuer permits insertion naming a deleted birth. A certified trace creates a birth, deletes it, then inserts a replacement anchored to that birth. Its independent live-anchor language permits the replacement before the deletion, but rejects it after the deletion. Specification visibility requires the actual causal deletion-before-replacement order. No admitted word satisfies that requirement, for any representation-state domain or arbitration policy. The relevant implementation effectors nevertheless commute on the proved closed invariant domain. Thus representation invariants repair an implementation-order issue without repairing this issuance/specification mismatch. A successful claim must reconcile those two interfaces explicitly.

A Mechanization traceability

This appendix is optional. The main definitions and statements can be read without knowledge of Lean. These links identify checked source declarations at the reference commit. Some displays group related declarations or specialize a generic interface to concrete equality. All listed declaration names were checked against the built environment. The final certificate roots are audited in the theorem ledger for standard Lean axioms.

Reference 1.1. [UpdateSignature.lean](#):

```
Sal.MRDTs.Foundation.Op
Sal.MRDTs.Foundation.UpdateSig.commutes
```

[RestrictedReplay.lean](#):

```
Sal.MRDTs.Paper1.OperationPolicy
Sal.MRDTs.Paper1.OperationPolicy.lift_rc_iff
```

[HistoricalReplay.lean](#):

```
Sal.MRDTs.Foundation.distinctOps
```

Reference 1.2. [GuardedReplay.lean](#):

```
Sal.MRDTs.Paper1.GuardedReplay.Laws
```

Reference 1.3. [InvariantOrder.lean](#):

```
Sal.MRDTs.Paper1.InvariantOrder.Commutates
Sal.MRDTs.Paper1.InvariantOrder.Closed
```

Reference 1.4. [InvariantOrder.lean](#):

```
Sal.MRDTs.Paper1.InvariantOrder.order
Sal.MRDTs.Paper1.InvariantOrder.order_universal
```

[RestrictedReplay.lean](#):

```
Sal.MRDTs.Paper1.paperOrder
```

[Replay.lean](#):

```
Sal.MRDTs.Foundation.listPermOf
Sal.MRDTs.Foundation.respects
```

Reference 1.5. [Certificates.lean](#):

```
Sal.MRDTs.MintHonest
Sal.MRDTs.Issuance
Sal.MRDTs.IssuedStep
```

Sal.MRDTs.MintCertifiedReach
Sal.MRDTs.MintCertifiedReachV

[Correctness.lean](#):

Sal.MRDTs.CertifiedExecution

[CertifiedReplay.lean](#):

Sal.MRDTs.Paper1.CertifiedReplay.Scope
Sal.MRDTs.Paper1.CertifiedReplay.Ready
Sal.MRDTs.Paper1.CertifiedReplay.Legal

Reference 1.6. [InvariantReplay.lean](#):

Sal.MRDTs.Paper1.InvariantReplay.Laws
Sal.MRDTs.Paper1.InvariantReplay.PolicyLaws
Sal.MRDTs.Paper1.InvariantReplay.RestrictedLaws

[CertifiedPolicy.lean](#):

Sal.MRDTs.Paper1.CertifiedReplay.Commutates

Reference 1.7. [InvariantReplay.lean](#):

Sal.MRDTs.Paper1.InvariantReplay.convergence_on

Reference 1.8. [HistorySpec.lean](#):

Sal.MRDTs.Paper1.HistorySpec
Sal.MRDTs.Paper1.HistoryMachine.toSpec

[SpecificationVisibility.lean](#):

Sal.MRDTs.Paper1.HistorySpec.Commutates

[ProjectedCriterion.lean](#):

Sal.MRDTs.Paper1.projectedLabels
Sal.MRDTs.Paper1.projectedSpecVisibility

Reference 1.9. [InvariantOrder.lean](#):

Sal.MRDTs.Paper1.InvariantOrder.VersionsRA
Sal.MRDTs.Paper1.InvariantOrder.versions_universal

[EventBridge.lean](#):

Sal.MRDTs.Paper1.EventVersionsSpecificationRA
Sal.MRDTs.Paper1.EventVersionsSpecificationRA.heads
Sal.MRDTs.Paper1.EventSequentialSimulation

[ProjectedCriterion.lean](#):

Sal.MRDTs.Paper1.ProjectedExceptionSpecificationRALinearizable

[RALinearizability.lean](#):

Sal.MRDTs.Paper1.RALinearizable

Reference 2.1. [Execution.lean](#):

Sal.MRDTs.Configuration
Sal.MRDTs.Step

Reference 2.2. [Execution.lean](#):

Sal.MRDTs.IsGCA

Reference 2.3. [StoreInvariant.lean](#):

`Sal.MRDTs.gca_events_of_storeInv`

Reference 2.4. [VirtualMergeBase.lean](#):

`Sal.MRDTs.virtualMergeBaseState`

`Sal.MRDTs.vfoldAux`

[Execution.lean](#):

`Sal.MRDTs.StepV`

Reference 3.1. [AbstractMerge.lean](#):

`Sal.MRDTs.Paper1.AbstractMRDT.Representation`

[GuardedRawJoin.lean](#):

`Sal.MRDTs.Paper1.AbstractMRDT.Raw.Unique`

Reference 3.2. [MetadataDependencies.lean](#):

`Sal.MRDTs.Paper1.AbstractMRDT.MetadataDependencies`

`Sal.MRDTs.Paper1.AbstractMRDT.MetadataDependencies.Closed`

`Sal.MRDTs.Paper1.AbstractMRDT.MetadataDependencies.Past`

`Sal.MRDTs.Paper1.AbstractMRDT.MetadataDependencies.closed_diff_of_max`

Reference 3.3. [GuardedRawJoin.lean](#):

`Sal.MRDTs.Paper1.AbstractMRDT.Raw.Context`

`Sal.MRDTs.Paper1.AbstractMRDT.Raw.MergeVCs`

[RestrictedReplay.lean](#):

`Sal.MRDTs.Paper1.paperOrder`

Reference 3.4. [GuardedRawJoin.lean](#):

`Sal.MRDTs.Paper1.AbstractMRDT.Raw.PeelChoice`

`Sal.MRDTs.Paper1.AbstractMRDT.Raw.ReplaySupply`

Reference 3.5. [GuardedRawJoin.lean](#):

`Sal.MRDTs.Paper1.AbstractMRDT.Raw.join_at_sizes`

`Sal.MRDTs.Paper1.AbstractMRDT.Raw.representationJoin_of_vcs`

Reference 3.7. [CertifiedClosedExecution.lean](#):

`Sal.MRDTs.Paper1.CertifiedClosedExecution.ClosedJoinAt`

`Sal.MRDTs.Paper1.CertifiedClosedExecution.canonicalConfig_of_execution`

`Sal.MRDTs.Paper1.CertifiedClosedExecution.virtualMergeBaseState_canonical`

[CertifiedRGAEExecution.lean](#):

`Sal.MRDTs.Paper1.CertifiedRGAEExecution.Embedded.represented_of_canonical`

`Sal.MRDTs.Paper1.CertifiedRGAEExecution.Embedded.canonical_of_represented`

`Sal.MRDTs.Paper1.CertifiedRGAEExecution.Embedded.closedJoinAt`

[SetRelativeReplay.lean](#):

`Sal.MRDTs.Foundation.loOn`

`Sal.MRDTs.Foundation.IsCanonicalState`

Reference 1. [SequentialSimulation.lean](#):

`Sal.MRDTs.Paper1.SequentialSimulation.sound`

[RALinearizability.lean](#):

`Sal.MRDTs.Paper1.ra_of_canonical_total`

Reference 2. [CommutationBridge.lean](#):

`Sal.MRDTs.Paper1.CommutationCompatibility`

`Sal.MRDTs.Paper1.projectedSpecVisibility_sub_paperOrder`

Reference 4.2. [ScopedHistoryBridge.lean](#):

`Sal.MRDTs.Paper1.AbstractMRDT.ExecutionHistoryAdequacy`

Reference 4.3. [ScopedHistoryBridge.lean](#):

`Sal.MRDTs.Paper1.AbstractMRDT.versions_of_scoped_history`

`Sal.MRDTs.Paper1.AbstractMRDT.ScopedVCConditions.executionsV`

[GuardedHistoryBridge.lean](#):

`Sal.MRDTs.Paper1.AbstractMRDT.Guarded.versions_of_scoped_history`

Reference 4.1. [CriterionCounterexample.lean](#):

`Sal.MRDTs.Paper1.CriterionCounterexample.mutant_rawRA`

`Sal.MRDTs.Paper1.CriterionCounterexample.criteria_differ_on_reachable_mutation`

Reference 4.4. [AnchoredQueueCausality.lean](#):

`Sal.MRDTs.Paper1.AnchoredQueue.removed_births_causal_downset`

[AnchoredQueueCertificate.lean](#):

`Sal.MRDTs.Paper1.AnchoredQueue.History.stored_publicWitness`

`Sal.MRDTs.Paper1.AnchoredQueue.History.correctV`

[AnchoredQueue.lean](#):

`Sal.MRDTs.Paper1.AnchoredQueue.fifoLegal`

`Sal.MRDTs.Paper1.AnchoredQueue.CanIssue`

[EmbedRGA.lean](#):

`Sal.MRDTs.Instances.EmbedRGA.eCoord`

[Code.lean](#):

`Sal.EmbedRGA.unaryEnc`

Reference 4.5. [CertifiedFugueInvariantObstruction.lean](#):

`Sal.MRDTs.Paper1.CertifiedFugueInvariantObstruction.certified_failure`

`Sal.MRDTs.Paper1.CertifiedFugueInvariantObstruction.implementation_spec_separation`